SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1710**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Deepika S / 1921260057 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. **Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.**

Answer:

Problem formulation

State: (x, y), where x = gallons in the 4-gallon jug (0 to 4) and y = gallons in the 3-gallon jug (0 to 3).

Initial state: (0, 0). Goal state: (2, y), i.e. exactly 2 gallons in the 4-gallon jug.

Actions (production rules): fill the 4-gallon jug, fill the 3-gallon jug, empty the 4-gallon jug, empty the 3-gallon jug, pour from the 4-gallon jug into the 3-gallon jug until it is full or the 4-gallon jug is empty, and pour from the 3-gallon jug into the 4-gallon jug until it is full or the 3-gallon jug is empty.

Path cost: each action costs 1.

Method

Breadth-First Search is used because every action has the same cost, so the first solution found has the fewest steps.

Solution

Step	Action	State (x, y)	Rule applied
0	Start	(0, 0)	Initial state
1	Fill the 3-gallon jug	(0, 3)	Fill y
2	Pour 3-gallon jug into 4-gallon jug	(3, 0)	Pour y into x
3	Fill the 3-gallon jug again	(3, 3)	Fill y
4	Pour into 4-gallon jug until it is full (1 gallon moves)	(4, 2)	Pour y into x
5	Empty the 4-gallon jug	(0, 2)	Empty x
6	Pour the 3-gallon jug into the 4-gallon jug	(2, 0)	Pour y into x (goal)

Alternative path (also 6 steps): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3).

Interpretation

After 6 actions the 4-gallon jug holds exactly 2 gallons, so the goal state (2, 0) is reached. Six actions is the minimum number of steps for this problem.

2. **Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.**
   i. What are the percept's for this agent?

ii. Characterize the operating environment.

iii. What are the actions the agent can take?

iv. How can one evaluate the performance of the agent?

v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

i. Percepts

- Camera images (stereo and panoramic) of terrain, rocks and soil
- Spectrometer and chemical sensor readings from rock and soil samples (mineral composition, moisture, organic traces)
- Position, wheel odometry, wheel slip and tilt/orientation (IMU) readings
- Obstacle and slope information from lidar or stereo vision
- Temperature, dust and wind readings, and Sun position
- Battery level and internal health status of instruments and motors
- Commands and updates received from the control centre on Earth

ii. Operating environment

Property	**Characterization**
Observable	Partially observable: sensors see only the nearby terrain
Agents	Single agent (other rovers are not interacting)
Deterministic	Stochastic: wheel slip, loose soil, dust and weather are uncertain
Episodic	Sequential: current moves affect later power and position
Static	Dynamic: dust storms, temperature and light change over time
Discrete	Continuous: continuous space, time and sensor values
Known	Partly unknown: terrain and samples are not fully mapped

iii. Actions

- Move forward or backward, turn left or right, and stop
- Navigate around obstacles and choose a new path
- Deploy the robotic arm, drill or scoop to collect a sample
- Run analysis instruments on a sample
- Capture images and store data
- Transmit data to Earth and receive commands
- Orient solar panels, recharge and enter sleep mode to conserve energy

iv. Performance measure

- Number and scientific quality of samples collected and analysed
- Area of Mars surface explored
- Accuracy and amount of data successfully sent to Earth
- Safety: no collisions, falls or getting stuck

- Efficient use of energy and time, and length of mission life

v. Suitable agent architecture

A utility-based agent with a learning component, built on a model-based reflex layer, is most suitable. Reasons:

- The environment is partially observable, so the agent must keep an internal model of the terrain and its own state.
- There are several goals that conflict (explore more, collect more samples, save energy, stay safe), so a utility function is needed to choose the best trade-off.
- Communication with Earth is delayed, so the rover must decide on its own and learn from new terrain.
- Simple reflex rules (stop at a cliff, stop on low battery) give an immediate safety response.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:

Problem formulation (incremental)

Component	**Definition**
States	Any arrangement of k queens ($0 \le k \le 8$), one in each of the leftmost k columns, with no queen attacking another
Initial state	Empty 8 × 8 board (no queens)
Actions	Place a queen in the leftmost empty column, in a row where it is not attacked by any queen already placed
Transition model	Returns the board with the new queen added
Goal test	8 queens are on the board and none attacks another (no two share a row, column or diagonal)
Path cost	Not relevant (only the final arrangement matters); each placement may be taken as cost 1

Why the given configuration is not a solution

Two queens at (row r1, column c1) and (row r2, column c2) attack each other along a diagonal when $|r1 - r2| = |c1 - c2|$. The queen in column 1 and the queen in column 8 lie on the same diagonal, so they attack each other and the goal test fails.

Search space and strategies

- If any 8 of the 64 squares may be chosen, the space has $64C8 \approx 4.4 \times 10^9$ states. Placing one queen per column gives $8^8 \approx 16.7$ million. Placing queens only on safe squares reduces this to just 2,057 states.
- Depth-First Search with backtracking is the most suitable uninformed method: every solution lies at depth 8, and the search needs memory only for the current path. When a

column has no safe row, the search backtracks to the previous column and tries the next row.

- Breadth-First Search also works but must store a very large frontier, so it needs far more memory.
- For a complete-state formulation (all 8 queens on the board, then move them), hill climbing or the min-conflicts heuristic can be used.

A valid solution

Rows of the queens in columns 1 to 8: (1, 5, 8, 6, 3, 7, 2, 4). No two queens share a row, and for every pair of columns the row difference differs from the column difference, so no two queens share a diagonal. This satisfies the goal test.

Interpretation

Because the incremental formulation rejects attacked squares at every step, it avoids exploring invalid boards, and backtracking DFS finds a solution after exploring only a small part of the space.

4. **A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.**

Answer:

Type of agent

A goal-based problem-solving agent with a utility-based choice of cab is suitable. The goal is to reach the destination. The agent formulates the problem (initial state = pickup location, goal = drop location), searches for a route, and then uses a utility function over fare, waiting time, travel time and comfort to select the cab type that best matches the customer's preference. It is a goal-based agent because it acts to achieve a stated goal, and utility-based because several cab types can reach the goal and the best one must be chosen.

Problem formulation

- Initial state: customer at the pickup location. Goal: customer at the destination.
- Actions: select a cab type, book, travel along a road segment, cancel.
- Path cost: fare, travel time and waiting time (weighted by the customer's comfort preference).

Pseudocode

```
function OLA-CAB-AGENT(percept) returns action
  persistent: seq (action sequence, initially empty), state, goal, problem
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
     goal ← FORMULATE-GOAL(state)            // destination entered by customer
     problem ← FORMULATE-PROBLEM(state, goal)
     seq ← SEARCH(problem)                   // route search from pickup to drop
     if seq = failure then return NO-ROUTE-AVAILABLE
  action ← FIRST(seq);  seq ← REST(seq)
  return action

procedure BOOK-CAB(pickup, drop, preference)
```

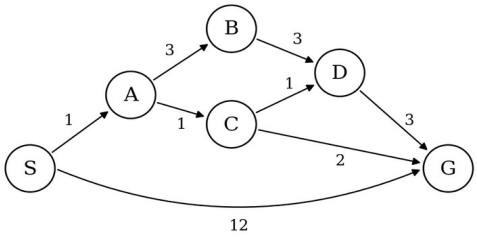
```
1.  Input pickup, drop and comfort preference
2.  distance, route ← FIND-ROUTE(pickup, drop)
3.  cabs ← GET-AVAILABLE-CABS(pickup)          // mini, micro, sedan, shared, prime ...
4.  best ← NULL;  bestUtility ← −∞
5.  for each cab in cabs do
6.      fare ← CALCULATE-FARE(cab, distance)
7.      eta  ← ESTIMATE-ARRIVAL(cab, pickup)
8.      comfort ← COMFORT-SCORE(cab.type)
9.      U ← w1*comfort − w2*fare − w3*eta
10.      if U > bestUtility and cab satisfies preference then
11.          best ← cab;  bestUtility ← U
12.  if best = NULL then display "No cab available"; stop
13.  customer confirms best  → book best
14.  while ride is not complete do TRACK(best.location)
15.  if best.location = drop then end ride, take payment, take feedback
end procedure
```

Interpretation

The agent always returns a booking for the cab type with the highest utility that meets the customer's comfort choice, and ends when the customer reaches the destination.

5. **A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.**

   The delivery network is represented as a weighted graph:



Answer:

Problem understanding

Edges (cost): S → A (1), S → G (12), A → B (3), A → C (1), B → D (3), C → D (1), C → G (2), D → G (3). UCS expands the node with the lowest path cost g(n) from the frontier (priority queue) and applies the goal test when a node is removed from the frontier. Ties are broken alphabetically.

Solution process

Step	Node expanded (g)	Frontier after expansion (node : g)	Remark
1	S (0)	A:1, G:12	Start
2	A (1)	C:2, B:4, G:12	B = 1+3, C = 1+1
3	C (2)	D:3, B:4, G:4	D = 2+1; G improved from 12 to 4 via C
4	D (3)	B:4, G:4	G via D = 6, not better than 4
5	B (4)	G:4	D via B = 7, already expanded
6	G (4)	—	Goal removed from frontier: stop

Order of expansion: S, A, C, D, B, G.

Path	Cost	Result
S → A → C → G	1 + 1 + 2 = 4	Least cost
S → A → C → D → G	1 + 1 + 1 + 3 = 6	
S → A → B → D → G	1 + 3 + 3 + 3 = 10	
S → G	12	

Interpretation

The least-cost path from S to G is S → A → C → G with a total cost of 4 (fuel + time units). The direct route S → G costs 12, three times more. Since all edge costs are positive, UCS is complete and optimal, so the path found is guaranteed to be the cheapest.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem